APOSTILA DE COMPILADORES

AULA 16

SINTAXE, SEMÂNTICA E AVALIAÇÃO DE EXPRESSÕES

Atributos • definições dirigidas por sintaxe • dependências • ações semânticas

Da AST semanticamente analisada à representação intermediária

Ciência da Computação — 2026/2
Professor: Alex Torquato Souza Carneiro
Projeto integrador: compilador da linguagem MINIC

Apresentação e organização da aula

Esta aula estabelece a ponte entre a estrutura sintática já representada pela AST e a próxima etapa do compilador: a representação intermediária (IR). A ideia central é associar regras de significado às produções da gramática, acompanhar de onde vêm as informações necessárias e definir em que ordem elas podem ser calculadas.

O conteúdo retoma a tradução dirigida por sintaxe e aprofunda sua aplicação à avaliação de expressões. O foco não é ainda implementar um gerador completo de IR: ao final, o estudante deverá compreender como atributos e ações semânticas podem transportar tipos, valores, referências a símbolos e estruturas de saída, preparando a tradução da AST para uma forma intermediária.

> **Fronteira com a Aula 17**
> Nesta aula veremos a motivação e uma prévia curta de representação intermediária para contextualizar os atributos. Código de três endereços em profundidade, temporários, rótulos, quádruplas, triplas e a tradução sistemática de comandos pertencem à aula seguinte. A Aula 15 encerrou as regras de análise semântica; a Aula 16 estuda mecanismos gerais de avaliação e tradução.

Objetivos de aprendizagem

- Explicar como uma gramática pode ser enriquecida com atributos e regras semânticas.
- Distinguir atributos sintetizados de herdados e reconhecer o fluxo de informação na árvore.
- Construir grafos de dependência e derivar ordens de avaliação válidas.
- Identificar definições S-atribuídas e L-atribuídas e relacioná-las a estratégias de parsing.
- Diferenciar uma definição dirigida por sintaxe (SDD) de um esquema de tradução (SDT) com ações posicionadas.
- Avaliar expressões respeitando estrutura, precedência, associatividade e as regras do MINIC.
- Explicar como resultados da análise semântica podem alimentar uma futura geração de IR.

Roteiro

Parte	Questão orientadora
1. Relação sintaxe–semântica	O que a gramática reconhece e o que as regras semânticas calculam?
2. Atributos	Como os dados sobem pela árvore ou chegam a um nó a partir do contexto?
3. Dependências e avaliação	Que informação precisa existir antes de uma regra ser executada?
4. SDDs/SDTs	Como organizar regras e ações para diferentes estratégias de parsing?
5. Expressões e MINIC	Como tipos, valores e resultados traduzidos percorrem uma expressão?

1. Da sintaxe ao significado

A sintaxe descreve as formas de programa admitidas: quais tokens podem aparecer e como se agrupam em declarações, comandos e expressões. A semântica define como interpretar essas estruturas. Uma expressão pode ser sintaticamente válida e ainda assim conter um identificador não declarado ou uma operação incompatível.

Camada	Pergunta respondida	Exemplo
Sintática	A sequência obedece à gramática?	a + * b é malformada.
Semântica estática	A estrutura válida respeita regras de nomes, tipos e contexto?	true + 3 pode ser rejeitada por incompatibilidade.
Semântica dinâmica	O que acontece quando o programa é executado?	O valor de uma divisão ou de um índice variável pode depender da execução.

Uma gramática livre de contexto por si só reconhece estrutura, mas não determina o tipo de uma expressão, não associa um uso de identificador à declaração visível e não calcula um valor. Para expressar tais propriedades, uma definição dirigida por sintaxe (SDD) associa atributos às ocorrências dos símbolos e equações semânticas às produções.

```
Produção:   E → E₁ + T
Regra:       E.tipo = tipoComum(E₁.tipo, T.tipo)
             E.valor = E₁.valor + T.valor     (quando a avaliação concreta é
desejada)
```

A mesma produção pode ter mais de uma regra: por exemplo, sintetizar o tipo e construir um objeto que represente a operação. Em um compilador, porém, calcular o valor concreto em compilação não é sempre apropriado. Para uma expressão com variáveis, costuma-se calcular propriedades estáticas ou construir uma representação do cálculo, deixando a execução do valor para outra fase.

SDD não é sinônimo de “executar o parser”

A SDD é uma especificação declarativa de atributos e regras. Ela pode ser avaliada depois que a árvore foi construída ou implementada durante a análise sintática, desde que a ordem de execução respeite as dependências.

2. Atributos sintetizados e herdados

2.1 Atributos sintetizados

Um atributo sintetizado em um nó é calculado a partir dos atributos dos filhos e, possivelmente, dos dados locais da ocorrência, como o lexema de um token. O fluxo característico vai de baixo para cima na árvore. É adequado para resultados que uma subárvore consegue determinar por si, como o tipo de uma expressão ou o nó AST que representa uma operação.

```
E → E₁ + T       E.tipo = tipoAritmetico(E₁.tipo, T.tipo)
                  E.ast  = Binary(+, E₁.ast, T.ast)
T → num           T.tipo = int
                  T.ast  = Literal(num.lexema, int)
```

No exemplo, as informações das subexpressões E_1 e T tornam-se disponíveis para calcular os atributos da expressão-pai E. A regra de tipo deve usar a política definida para o MINIC: por exemplo, operandos numéricos compatíveis podem resultar em int ou float conforme os tipos dos operandos e as conversões autorizadas.

2.2 Atributos herdados

Um atributo herdado de uma ocorrência pode depender de atributos do pai, de informações do ambiente de compilação ou de atributos de irmãos que já estejam disponíveis. Ele transporta contexto para baixo ou ao longo da árvore. Exemplos: o tipo declarado que deve ser associado a cada nome de uma declaração; o tipo de retorno esperado dentro do corpo de uma função; o escopo no qual um identificador deve ser resolvido.

```
D → T L           L.inh = T.tipo
L → id            inserir(id.lexema, L.inh)
L → L₁ , id       L₁.inh = L.inh
                  inserir(id.lexema, L.inh)
```

Aqui, o tipo sintetizado por T é passado como contexto herdado para L e, na produção recursiva, propagado à lista. A regra pressupõe que T.tipo já esteja disponível antes de processar L. A forma exata da gramática e as ações variam; o ponto didático é explicitar o fluxo de informação.

Aspecto	Sintetizado	Herdado
Direção típica	Filhos → pai	Pai/irmãos anteriores → nó
Uso frequente	Tipo/valor de expressão; AST; lista de instruções	Tipo declarado; ambiente; contexto de função/laço
Exemplo MINIC	tipo de a + b a partir de tipo(a) e tipo(b)	retornoEsperado propagado ao corpo da função
Avaliação comum	Pós-ordem; resultados sobem	Pré-ordem ou percurso esquerda-para-direita

Dica para classificar

Pergunte de quais informações a regra depende. Se o resultado do nó vem das subárvores, tende a ser sintetizado. Se o nó recebe uma condição ou ambiente decidido fora dele, tende a ser herdado. A direção visual ajuda, mas a dependência formal é o critério.

3. Definições dirigidas por sintaxe e dependências

Uma SDD especifica atributos e regras para cada produção. Para uma árvore concreta, as variáveis da regra tornam-se instâncias de atributos: E.tipo em um nó específico é diferente de E.tipo em outro nó. O grafo de dependência representa essas instâncias e as precedências impostas pelas regras.

3.1 Construção do grafo

1. Crie um vértice para cada instância de atributo necessária na árvore, por exemplo, o tipo do filho esquerdo e o tipo do pai.
2. Para cada equação que calcula um atributo destino a partir de um atributo fonte, desenhe uma aresta da fonte para o destino.
3. Repita para todas as regras aplicáveis à árvore concreta.
4. Verifique se o grafo tem ciclos; se for acíclico, qualquer ordenação topológica respeita as dependências.

```
E.tipo = combine(E₁.tipo, T.tipo)

E₁.tipo ──┐
          ├──> E.tipo
T.tipo  ──┘
```

A seta significa “precisa ser conhecido antes de calcular”; não significa igualdade ou transferência literal. Se uma regra usa uma constante, um lexema ou o ambiente global, esses elementos podem ser vistos como dados já disponíveis.

3.2 Ordem de avaliação

Uma ordem válida de avaliação é qualquer sequência que coloque cada atributo depois de todas as instâncias das quais ele depende. Para um grafo acíclico, obtém-se essa sequência por ordenação topológica. Pode haver mais de uma ordem correta; a gramática não exige uma execução única, apenas que toda dependência seja respeitada.

Para uma definição inteiramente sintetizada, uma travessia pós-ordem da árvore calcula primeiro os filhos e depois o pai. Para atributos herdados em uma definição L-atribuída, uma travessia da esquerda para a direita pode calcular o contexto de cada filho antes de percorrê-lo. Para SDDs gerais, não se presume que um simples percurso fixo seja suficiente: as dependências da árvore precisam ser consideradas.

3.3 Ciclos e inconsistências

Se houver um ciclo de dependências, não existe ordenação topológica para aquela árvore; as regras não determinam um cálculo sequencial finito simples. Um ciclo pode indicar uma especificação malformada ou uma propriedade que exigiria outro mecanismo, como resolução por ponto fixo, que está fora do escopo desta aula. Em uma SDD não circular, o grafo de cada árvore de derivação deve ser acíclico.

Exercício breve

Considere P → A B, A.x = entrada e B.y = A.x + 1. Desenhe os nós A.x e B.y, acrescente a aresta e explique por que B.y não pode ser calculado antes de A.x. Se também definirmos A.x = B.y − 1, o que ocorre?

4. Esquemas S-atribuídos e L-atribuídos

4.1 S-atribuída

Uma SDD é S-atribuída quando todos os seus atributos são sintetizados. Como os valores necessários vêm dos filhos, a avaliação combina naturalmente com uma travessia pós-ordem. Em um parser LR, a redução de uma produção ocorre depois de reconhecer seu lado direito; os valores semânticos dos símbolos reduzidos podem ser usados para calcular e empilhar o atributo da cabeça.

```
E → E₁ + T   { E.ast = Binary(+, E₁.ast, T.ast);
               E.tipo = tipoAritmetico(E₁.tipo, T.tipo) }
E → T          { E.ast = T.ast; E.tipo = T.tipo }
T → id         { T.ast = Identifier(id.lexema);
               T.tipo = resolverTipo(id.lexema) }
```

A classe S-atribuída é útil para expressões e construção de AST. Nem toda consulta contextual precisa ser incorporada diretamente em uma regra S-atribuída; pode-se fazer depois uma travessia semântica que consulta uma tabela de símbolos. A classificação depende dos atributos usados na definição, não apenas da ferramenta empregada.

4.2 L-atribuída

Uma SDD é L-atribuída se, em cada produção $A \to X_1\ X_2\ \dots\ X_n$, cada atributo herdado de X_i depender apenas de atributos herdados de A e de atributos associados aos símbolos $X_1,\dots,X_{i-1}$ que já aparecem à esquerda, além de dados locais permitidos. A restrição impede que o atributo dependa de um símbolo à direita que ainda não foi processado.

Essa propriedade permite uma avaliação da esquerda para a direita: antes de visitar um filho, calcula-se o contexto recebido por ele com base na cabeça e nos irmãos anteriores. Todas as SDDs S-atribuídas são L-atribuídas (não há atributos herdados que violem a condição), mas nem toda L-atribuída é S-atribuída.

```
Decl → Tipo Lista
Tipo → int        Tipo.tipo = INT
Decl.inh → Lista   Lista.tipo = Tipo.tipo
Lista → id        declarar(id, Lista.tipo)
Lista → Lista₁ , id
                  Lista₁.tipo = Lista.tipo
                  declarar(id, Lista.tipo)
```

Na implementação, a atribuição a uma tabela de símbolos é um efeito lateral; o tipo propagado para Lista é o atributo herdado central do exemplo. Uma implementação robusta pode separar a especificação da política concreta de armazenamento, evitando que a ação dependa acidentalmente de uma ordem de visita não explicitada.

Classe	Atributos permitidos	Ordem típica	Relação com parsing
S-atribuída	Somente sintetizados	Pós-ordem	Natural em reduções LR
L-atribuída	Sintetizados e herdados sob a restrição à esquerda	Esquerda para direita	Adequada a parsers descendentes recursivos
SDD geral	Pode incluir outras dependências, se acíclicas na árvore	Ordenação topológica do grafo	Pode exigir estratégia mais geral ou múltiplas passagens

Atenção à terminologia

S-atribuída e L-atribuída classificam definições dirigidas por sintaxe. “Ascendente” e “descendente” classificam estratégias de análise sintática. Há afinidades de implementação, mas não são sinônimos nem imposições absolutas.

5. Definições dirigidas por sintaxe e ações semânticas

Uma definição dirigida por sintaxe descreve relações entre atributos, geralmente como equações associadas às produções. Um esquema de tradução dirigida por sintaxe (SDT) insere ações executáveis no corpo das produções. A posição da ação no lado direito estabelece quando ela pode ocorrer durante o parsing.

```
Produção com ação no final:
E → E₁ + T  { E.ast = Binary(+, E₁.ast, T.ast) }

Ação intermediária, ilustrando propagação de contexto:
D → T  { contexto.tipo = T.tipo }  L  { concluirDeclaracao() }
```

A ação no final costuma ser adequada quando depende de todos os símbolos do lado direito. Uma ação intermediária pode preparar contexto para símbolos seguintes; nesse caso, o parser deve assegurar que os valores lidos já existam. Para uma SDD S-atribuída em análise LR, as ações aparecem naturalmente ao fim da produção. Em parsers descendentes, ações podem ocorrer antes de chamadas recursivas para transmitir atributos herdados.

5.1 Tipos de ações em um compilador

Ação	Exemplo de uso	Cuidado
Construção de objeto	Criar nó Binary ou Call da AST.	Preservar posições e associatividade.
Cálculo de atributo	Calcular tipo de expressão a partir dos filhos.	Aplicar a matriz de tipos especificada.
Propagação de contexto	Passar tipo de retorno esperado ao corpo.	Disponibilizar o contexto antes da visita.
Atualização de ambiente	Abrir/fechar escopo, inserir declaração.	Controlar escopo e evitar efeitos duplicados.
Emissão de tradução	Acumular representação de uma construção.	Manter ordem e evitar saída parcial incorreta.

Na etapa atual do MINIC, o parser constrói a AST e a análise semântica resolve nomes, tipos e contextos. A partir de uma AST aprovada, a etapa de IR poderá percorrer os nós e emitir instruções. A separação por fases não proíbe esquemas de tradução integrados; é uma decisão de arquitetura que facilita testar cada responsabilidade e evita misturar análise com geração prematura.

> **Princípio de projeto**
> Uma ação semântica deve estar associada a uma dependência compreensível. Se uma ação cria código ou altera estado global sem que a ordem esteja clara, a definição fica frágil e difícil de testar.

6. Avaliação de expressões

6.1 Estrutura, precedência e associatividade

A avaliação estrutural segue a AST, não a ordem visual isolada dos operadores. A gramática ou as regras de precedência/associatividade determinam o agrupamento. Por exemplo, se multiplicação tem precedência maior que adição, a + b * c corresponde à árvore a + (b * c); com associatividade à esquerda, a - b - c corresponde a (a - b) - c.

```
a + b * c

      +
     / \
    a   *
       / \
      b   c
```

Parênteses explícitos podem alterar o agrupamento. Depois de construída a AST, uma visita em pós-ordem calcula atributos sintetizados: avalia primeiro os filhos e aplica a regra do operador no nó. Essa ordem estrutural não determina, por si só, todas as decisões da semântica operacional: curto-circuito, efeitos colaterais e ordem de avaliação dos operandos precisam ser definidos pela linguagem.

6.2 Tipo, valor e tradução são atributos diferentes

Informação	O que representa	Exemplo em 2 + 3
tipo	Classe de valor autorizada pelo sistema de tipos.	int
valor	Resultado concreto, se for possível e desejável calculá-lo.	5
AST	Estrutura abstrata que registra a operação.	Binary(+, Literal(2), Literal(3))
código/IR	Instruções que representam o cálculo para execução posterior.	Será produzido na etapa de IR.

É importante não confundir o tipo de uma expressão com seu valor. Em x + 3, o compilador pode determinar o tipo sem saber o valor de x. Também não é preciso reduzir a expressão a um número para traduzi-la: um gerador pode produzir uma representação do cálculo, preservando a operação e os operandos.

6.3 Regras de tipo em expressões MINIC

A Aula 15 fixou as regras estáticas do núcleo MINIC. Nesta aula, usamos essas regras como exemplo de atributos sintetizados. Literais fornecem tipos a partir do token; identificadores usam a referência ao símbolo resolvido; operadores verificam requisitos dos operandos e sintetizam o tipo do resultado. Um marcador ERROR pode propagar falhas já diagnosticadas sem gerar cascatas.

Forma	Atributos consultados	Resultado didático
Literal inteiro	lexema do token	tipo int
Identificador x	símbolo associado a x	tipo declarado do símbolo
a + b	tipos numéricos dos operandos	int se ambos int; caso misto, float com promoção conforme a regra
a < b	tipos numéricos e compatibilidade	bool
p && q	tipo bool de p e q	bool; erro se algum não for bool
v[i]	v deve ser vetor; i deve ser int	tipo do elemento
f(args)	assinatura e tipos/quantidade dos argumentos	tipo de retorno de f; void não fornece valor

6.4 Exemplo de anotação de expressão

Considere a expressão (n + 2) * 1.5, com n declarado int e literais 2:int e 1.5:float. A AST é determinada pela precedência e pelos parênteses. A avaliação dos atributos de tipo é sintetizada de baixo para cima.

```
              * : float
            /       \
     + : int?*        1.5 : float
    /       \
 n : int      2 : int

Para a operação interna: int + int → int.
Para a operação externa: int * float → float, com promoção int → float
no operando esquerdo, conforme a política de conversões da Aula 15.
```

O rótulo “int?” acima merece cuidado: o tipo da soma é int sob a matriz didática definida. Na operação externa, a promoção do valor da soma é necessária para compatibilizar os operandos; o tipo resultante é float. A anotação pode registrar tanto o tipo original do nó quanto a coerção requerida no contexto da operação.

6.5 Curto-circuito e efeitos

Para && e ||, a verificação estática examina os tipos dos operandos, mas uma execução em curto-circuito pode não avaliar o operando da direita: em p && q, se p for falso, q não precisa ser executado; em p || q, se p for verdadeiro, q pode ser omitido. Se expressões tiverem chamadas ou outros efeitos, a ordem importa. Por isso, a semântica da linguagem e a futura tradução devem preservar o comportamento especificado, sem assumir reordenação livre.

O grafo de dependência das regras de atributos não substitui a ordem operacional dos efeitos em tempo de execução. Um determina quando atributos podem ser calculados pelo compilador; o outro determina o comportamento do programa traduzido. Essa distinção evita confundir dependências de compilação com execução da expressão.

7. Ponte para a representação intermediária

A AST validada contém operações, identificadores já associados a símbolos e tipos conhecidos. A próxima etapa transforma essa estrutura em uma representação explícita para execução, análise ou geração de código de destino. Os atributos sintetizados podem carregar resultados parciais da tradução; atributos herdados podem transportar contexto, como rótulos de controle, convenções de chamada ou informação sobre destino. Esses usos serão formalizados na Aula 17.

Como prévia, uma expressão aritmética simples pode ser representada por instruções elementares, em vez de manter apenas a árvore. A sequência abaixo é ilustrativa e antecipa apenas a ideia; a forma canônica e os detalhes serão objeto da aula seguinte.

```
AST:  a + b * c

Ideia de tradução em passos:
 t1 = b * c
 t2 = a + t1
 resultado = t2
```

Os nomes t1 e t2 representam resultados intermediários temporários. O gerador precisa preservar a estrutura da AST, as conversões de tipo anotadas e a ordem de efeitos estabelecida pela linguagem. Para comandos como if e while, a tradução também precisa representar desvios de controle; isso requer rótulos e saltos, que não serão desenvolvidos aqui.

Entrada da geração	Origem na AST/semântica	Papel futuro
Operador e filhos	Nós da AST	Escolher operação da IR e operandos.
Tipo e coerções	Anotações semânticas	Selecionar operação/conversão compatível.
Símbolo resolvido	Tabela de símbolos/referência do nó	Determinar a variável/função designada.
Posição de origem	Tokens e nós da AST	Manter rastreabilidade e diagnósticos.
Contexto de controle	Estrutura de if/while/funções	Gerar fluxo de controle na representação.

Contrato entre etapas

A etapa de IR não deve ter de redescobrir se x foi declarado ou se uma soma é válida. Ela recebe uma AST validada e anotações semânticas confiáveis. Separar responsabilidades reduz duplicação e torna os testes mais localizados.

8. Percurso de avaliação: algoritmo conceitual

Para uma AST de expressões, um avaliador ou anotador pode ser organizado recursivamente. A função visita os filhos, consulta símbolos e aplica regras do nó. Em vez de executar a aritmética, o compilador pode retornar um registro com tipo, conversões e uma estrutura resultante.

```
analisarExpressao(no, ambiente):
    se no é literal:
        devolver {tipo: tipoDoToken(no), ast: no}
    se no é identificador:
        simbolo = resolver(no.nome, ambiente)
        se simbolo inexistente: diagnosticar; devolver ERROR
        devolver {tipo: simbolo.tipo, simbolo: simbolo}
    analisar os filhos conforme a regra da construção
    verificar requisitos e compatibilidade
    anotar o nó com tipo/conversões
    devolver o resultado sintetizado
```

Esse pseudocódigo é uma forma de organização; não substitui a especificação completa das regras de chamadas, vetores, operadores e contextos. O percurso precisa respeitar a avaliação contextual. Por exemplo, resolução de identificadores depende do ambiente correto; retornos dependem do tipo da função; break/continue dependem da profundidade de laço. Esses dados podem ser parâmetros (herdados) ou parte do estado controlado do visitante.

9. Aplicação orientada ao projeto MINIC

9.1 Dados que atravessam as etapas

Etapa/artefato	Informação principal	Como será utilizada
Scanner	Token, lexema, linha e coluna.	Identificar folhas da AST e localizar mensagens.
Parser	Estrutura sintática e precedência.	Construir AST coerente com a gramática.
Tabela de símbolos	Declarações, tipos, categorias e escopos.	Resolver identificadores e validar chamadas.
Análise semântica	Tipos, coerções permitidas, símbolos associados, diagnósticos.	Confirmar que o programa é traduzível segundo as regras.
AST anotada	Operadores, referências, tipos e posições.	Entrada da tradução para IR.
Representação intermediária	Operações normalizadas e fluxo explícito.	Otimização, interpretação ou geração de código.

9.2 Decisões de implementação recomendadas

- Manter gramática, regras de atributos e ações semânticas documentadas, mesmo quando a implementação usa visitantes em vez de ações embutidas no parser.
- Registrar para cada nó quais atributos são sintetizados e quais dados de contexto são recebidos.
- Separar a construção da AST da validação semântica quando isso simplificar o parser e permitir testes por fase.
- Usar resultados tipados e explícitos (por exemplo, tipo + conversões + referência ao símbolo) em vez de recomputar informações em cada fase.
- Evitar geração de IR antes de a construção estar semanticamente validada, exceto quando a arquitetura definir recuperação segura.
- Escrever testes para as regras de avaliação e dependência, além de testes ponta a ponta.

10. Atividades de consolidação

Atividade A — classificar atributos

Para cada informação, diga se é naturalmente sintetizada, herdada ou depende de decisão arquitetural. Justifique a dependência: (a) tipo de a+b; (b) ambiente léxico em que se resolve x; (c) AST de chamada f(1); (d) tipo de retorno esperado no corpo de uma função; (e) lista de instruções resultante da tradução de uma expressão.

Atividade B — grafo e ordem

Considere E → E_1 + T com E.tipo = combine(E_1.tipo,T.tipo), E.ast = Binary(+,E_1.ast,T.ast) e E.constante = E_1.constante + T.constante quando ambos são constantes. Desenhe as dependências relevantes para cada atributo. Dê uma ordem válida e explique quando constante não pode ser calculado.

Atividade C — S ou L?

Classifique cada definição. (1) Todas as regras constroem ASTs apenas a partir dos filhos. (2) A declaração de uma lista de nomes recebe o tipo sintetizado pelo símbolo Tipo que está à esquerda. (3) Um atributo herdado de B depende do atributo de C, onde A → B C. Diga se o caso pode ser L-atribuído e por quê.

Atividade D — avaliação MINIC

Para n:int, r:float e ok:bool, anote os tipos de (n+2)*r, n<r, ok && (n>0), r[n] (supondo que r seja escalar) e f(n) quando f retorna void. Identifique conversões, resultados e erros semânticos. Não confunda erro estático com condição de execução.

Atividade E — desenho de interface

Proponha uma estrutura de resultado para visitar um nó de expressão e alimentar a IR: quais campos armazenar para tipo, coerções, referência a símbolo, posição e representação produzida? Explique quais campos são atributos sintetizados e quais informações seriam recebidas do contexto.

11. Síntese e checklist

A tradução dirigida por sintaxe associa regras de significado à estrutura gramatical. Atributos sintetizados levam resultados dos filhos ao pai; atributos herdados transportam contexto a partir do pai ou de irmãos anteriores. O grafo de dependência explicita precedências de cálculo, e uma ordenação topológica fornece uma avaliação válida quando o grafo é acíclico.

Definições S-atribuídas usam apenas atributos sintetizados e são naturalmente avaliadas em pós-ordem; definições L-atribuídas permitem atributos herdados sob uma restrição que possibilita processamento da esquerda para a direita. Ações semânticas materializam regras em pontos definidos do parsing, exigindo que dados usados já estejam disponíveis. Na avaliação de expressões, a AST define o agrupamento; atributos podem sintetizar tipo, coerções, valor constante ou tradução, que são resultados distintos.

- ☐ Sei separar sintaxe, semântica estática e comportamento em execução.
- ☐ Consigo identificar dependências de cada atributo e construir o grafo correspondente.
- ☐ Sei justificar uma ordem de avaliação e reconhecer ciclos.
- ☐ Consigo classificar uma SDD como S-atribuída ou L-atribuída.
- ☐ Sei explicar por que a posição de uma ação semântica importa.
- ☐ Distingo tipo, valor concreto, AST e representação intermediária.
- ☐ Entendo que a IR recebe informação semântica validada e será estudada em profundidade na Aula 17.

Referências

AHO, Alfred V.; SETHI, Ravi; ULLMAN, Jeffrey D. Compiladores: princípios, técnicas e ferramentas. Livro-texto fornecido para a disciplina. Capítulo 5, especialmente as Seções 5.1 e 5.2 (definições dirigidas por sintaxe, avaliação de atributos, grafos de dependência e classes S-atribuída/L-atribuída); Capítulo 5, Seção 5.3 (aplicações); Capítulo 6 (geração de código intermediário).